Plan: invoice anomalies, from simulation to MLOps

Written 2026-10-12. Two projects, one story. Project 01 plants anomalies and their honest look-alikes in a simulated accounts-payable operation and writes every one into an answer key. Project 02 learns to find them, with MLflow at the centre of an MLOps lifecycle. Each step is test first, and a step is done only when its check passes.

The references behind every choice are in the wiki note `/05-research/invoice-anomaly-mlops-repos.md`: six Kaggle gold-medal notebooks, Le Borgne and Bontempi's practical handbook on prequential validation, PyOD, a graph-based detection library, an AWS detection blueprint, the NeurIPS bank-account dataset suite, and the MLOps Zoomcamp rubric.

An anomaly here is always an error at one stage of the process: purchasing, the vendor master, receiving, billing or freight billing.

Charts: projects 01 and 02 draw their charts only in the monitoring stack, Prometheus and Grafana, fed by `prometheus_client`. Project 03 is the dashboard. Project 04 is on standby.

Part A. Project 01: a realistic, labelled world

A1. What an accounts-payable desk holds besides the invoice

Three-way match, the core control of accounts payable, compares the invoice with the purchase order and the goods receipt. A model needs the same. Project 01 exports, per run, under `out/erp/`:

Table	Holds
`vendors.csv`	each issuer: id, name, country, onboarding date, bank account on file, payment terms
`vendor_changes.csv`	every change to the vendor master: bank account, address, with date and who asked
`purchase_orders.csv`, `po_lines.csv`	order, issuer, date, item, quantity, agreed price, contract version
`contracts.csv`	agreed prices with their validity dates, including renewals
`receipts.csv`	what arrived, when, against which order (the goods receipt)
`shipments.csv`	mode, lane, weight, volume, dates
`rate_card.csv`	agreed freight and haulage rates per lane and container
`approval_policy.csv`	the approval limit per invoice, and who approves above it

Check: every invoice in the corpus joins to its issuer and, where it has one, to its order, receipt and shipment, with no orphans.

A2. What the invoice page must show

Addition	Why
the bank account to pay into	an account the vendor master does not hold is invisible without it
the sender address on the mail envelope	a bill sent from an address not on file is invisible without it

The reader extracts both. Check: the reader's score on the new fields is measured like the others, text and scans apart.

A3. The anomaly catalogue

Each anomaly is named in the ledger with the invoice it touches. Rates follow the sources; each kind is turned on by the busy and year profiles.

Kind	What happens	Pattern in the references	Source for the rate	Rate
`price_drift`	unit price above the contract in force	value out of line	APQC, erroneous payments	1.0% of goods
`quantity_over`	billed quantity above the goods receipt	value out of line	three-way-match practice	0.8% of goods
`round_total`	an "adjustment" lifts the total to round thousands	value out of line	Benford and round-number audit tests	0.4% of goods
`near_duplicate`	the same bill under a new number	repetition	APQC, IOFM duplicate payments	0.5%
`freight_overbill`	a charge outside the rate card	value out of line	Tompkins Ventures freight audits	7% of freight
`unmatched_vendor`	a vendor in the master with no order behind its bills	relations: bill to order to receipt	set, not sourced	2 to 3 vendors a year
`account_mismatch`	an account and a sender the vendor master does not hold	per-entity history (IEEE-CIS winner)	set, not sourced	0.2%
`split_billing`	one delivery billed in pieces, each under the approval limit	repetition and frequency	three-way-match practice	1.2% of goods over the limit

The first five exist; the last three are new.

A4. Honest look-alikes, the false positives a model must survive

Each anomaly has a twin that looks the same and is legitimate. The ledger marks it `lookalike_of: <kind>`.

Anomaly	Honest twin	What tells them apart
`price_drift`	a price increase under a renewed contract	`contracts.csv` holds the new price first
`quantity_over`	an over-delivery accepted at the dock	`receipts.csv` shows the extra units
`round_total`	a fixed fee: storage, a monthly service	the fee is in the rate card
`near_duplicate`	a monthly recurring charge of the same amount	a regular interval and a contract line
`freight_overbill`	a surcharge the rate card allows (demurrage, peak season)	the rate card lists it
`unmatched_vendor`	a real new supplier	onboarded in `vendors.csv`, with an order behind it
`account_mismatch`	a bank change announced and recorded first	`vendor_changes.csv` has it, from the known address
`split_billing`	partial deliveries of one order	each invoice ties to its own receipt

Rates: one twin for every anomaly, so a model that flags appearances alone scores half precision at best.

A5. Labels a model can read

File	One row per	Columns
`out/labels.csv`	invoice	number, issuer, issue date, anomaly kind or empty, look-alike-of or empty, delay in days until the anomaly would be discovered

The discovery delay follows the Handbook: labels arrive late in real life, so training never sees the most recent labels.

A6. Enough data, in time

Profile	Weeks	Invoices	Anomalies	Use
`busy`	8	about 820	about 40	the README corpus
`year`	52	about 5,300	about 250	training and testing
`drift`	26	about 2,700	rates shifted; unmatched vendors bill as freight	the monitoring scenario

Each runs with several seeds. One seed is held out and never trained on.

A7. Scans

Finish the scan measurement first: the target is under 1% of totals wrong and unflagged. The model learns from what the reader extracted, so reading errors are noise it must survive.

A8. Checks for Part A

- Every planted anomaly and twin is in `labels.csv`, and nothing else is.
- The quiet run still has its 102 invoices.
- Each rate lands within its band over the year profile.
- A model with the label leaked in scores perfectly; a model on shuffled labels scores at chance. Both are tests.

Part B. Project 02: finding them, with MLflow throughout

B1. Features, by the lessons of the references

Family	Examples	Reference
against the contract	price over the price in force; quantity over receipt; freight over rate card	three-way match
against the issuer's history	amount z-score per issuer; days since last invoice; invoices in the last 7 days; share of round totals	IEEE-CIS winner: score against the entity
against the vendor master	bank account equals the account on file; sender equals the known address; vendor age; has an order	vendor master checks
near duplicates	same issuer and amount within N days; text similarity of lines	duplicate-payment audits
digits	leading-digit deviation per issuer (Benford); trailing zeros	Benford tests
relations	issuer to order to receipt graph degree; orders without receipts	graph-based detection

Features are computed only from data available at the invoice date. A test fails if any feature reads the future.

B2. Models, from simplest to strongest

1. Rules alone, as the bar to beat.
2. Unsupervised: PyOD Isolation Forest and ECOD.
3. Supervised: logistic regression, then LightGBM and XGBoost, tuned.

4. A combined model: rules for the certain cases, the model for the rest, packaged as one MLflow `pyfunc`.

Resampling and class weights only inside validation folds.

B3. Validation and metrics

- Prequential validation by week, with the label delay, as Le Borgne and Bontempi do; month-grouped folds as the IEEE winner did.
- The headline: precision in the top k invoices per day, with k the number a person can check.
- Also: PR-AUC, recall per anomaly kind, false alarms per look-alike kind.
- The final score on the held-out seed only, reported once.

B4. MLflow, everywhere

Feature	Use
tracking server with a Postgres backend and an artifact store	every run, shared
`mlflow.data`	each training table logged with its seed, profile and hash
autolog and nested runs	the tuning search, one child per trial
`mlflow.evaluate` with custom metrics	top-k precision, per-kind recall, per-twin false alarms
artifacts	metric tables, SHAP values and feature importances as data, no chart files
signatures and input examples	the API refuses a malformed invoice
custom `pyfunc`	rules plus model, one artefact
model registry, aliases	`champion` serves; `challenger` must beat it on the held-out seed
`mlflow models build-docker`	the serving image
MLflow Projects (`MLproject`)	one command reproduces any run

B5. Serving

- A FastAPI service that loads `models:/invoice-anomaly@champion` and scores one invoice or a batch, with the reason codes behind each score.
- Batch scoring after each intake run, written beside the CSV for the payables panel (project 03) to show.

B6. Monitoring and retraining

- Feature drift and score drift computed per week, exported as Prometheus metrics; no HTML reports.
- The drift profile shifts the rates; unmatched vendors bill as freight.
- When top-k precision on newly labelled weeks falls below its threshold, or drift passes its threshold, a retraining flow runs; the new model becomes `challenger`, and is promoted only if it wins.
- Every chart lives in Grafana, on the Prometheus stack of project 01: scores, top-k precision per week, drift, retraining events.

B7. Orchestration

Prefect flows: generate, extract, build features, train, evaluate, register, promote, score, monitor. Each flow is idempotent and logged.

B8. Engineering practice, the Zoomcamp checklist

Item	How
unit tests	every feature and metric, test first
integration test	generate a small corpus, train, register, serve, score
linter and formatter	ruff
Makefile	`make data`, `make train`, `make serve`, `make test`
pre-commit	ruff and the fast tests
CI/CD	GitHub Actions: tests on every push, image build on main
reproducibility	pinned versions, one command per stage
cloud	Terraform for the tracking server, artifact bucket and the API; the same stack locally with Docker Compose

B9. Done when

- Every Zoomcamp criterion at full marks.
- Top-k precision, PR-AUC, per-kind recall and per-twin false alarms on the held-out seed, in the README.
- The drift scenario shown end to end: alert, retrain, challenger beats champion, promotion, the panel showing the new scores.

Order of work

#	Step	Depends on	State, 2026-10-12
1	Finish the scan score in 01	nothing	done: 0.4% wrong and unflagged
2	ERP export tables (A1)	nothing	done
3	Bank account and sender on the page, read back (A2)	2	done
4	The three new anomalies (A3)	2, 3	done
5	The look-alikes (A4)	4	done
6	`labels.csv` with discovery delay (A5)	5	done
7	Year and drift profiles, several seeds (A6)	6	done: `run.py dataset`
8	Leak and shuffle tests (A8)	7	moved to step 11, they need a model
9	Features with the no-future test (B1)	8	done: 20 features, 15 s a year
10	MLflow stack locally (B4)	nothing	done: SQLite tracking, datasets logged, `run.py ui`
11	Baselines and models, prequential validation (B2, B3)	9, 10	done: four models; 11 walk-forward folds; LightGBM 0.90 average precision held out. Leak and shuffle tests still to write
12	Registry, pyfunc, serving (B4, B5)	11	started: champion alias, promoted when it wins or when the champion reads other features; scores served to project 03 and to Prometheus
13	Monitoring in Prometheus and Grafana, and retraining (B6)	12	
14	Prefect flows (B7)	13	
15	CI, Makefile, pre-commit, Terraform (B8)	14	
16	Panel shows the scores (project 03)	12	
17	README, four documents, GIF, Upwork	all	